



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4306 · Design & Analysis of Algorithms · Fall 2026

Recursion and Recurrences

When the loops vanish, so does every counting rule — and an equation takes their place

Session 8 of 24 · Tuesday 13 October 2026 · 2:30 – 3:50 PM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 8 OF 24

Recursion and Recurrences

Tuesday, 13 October 2026 · 2:30 – 3:50 PM

LEARNING OBJECTIVES

- ✓ Explain why the obvious recursive Fibonacci is exponential, using its call tree.
- ✓ Distinguish recursion that splits into *disjoint* subproblems from recursion that *overlaps*.
- ✓ Translate a recursive Python function into a recurrence relation for its running time.
- ✓ Recognise the general divide-and-conquer form $T(n) = a \cdot T(n/b) + f(n)$ and identify a , b and f .

FROM SESSION 5

Sequences add, nests multiply, dependent nests need the series. Every one of those rules needs a **loop to count**. A recursive function has no visible iteration count at all, so today the toolkit we just built stops working and we need a different instrument.

1 · The Three-Line Disaster 20 min

The Fibonacci numbers are defined by $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$. Transcribed into Python, the definition becomes what may be the most beautiful and most catastrophic function in this course.

PY THE DEFINITION, TRANSCRIBED

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

It is correct. Every value it returns is right. Run it on $n = 35$ and you will wait; run it on $n = 50$ and you will not see the answer today. Before explaining why, let us make the function confess by counting its own calls.

PY INSTRUMENTED VERSION

```
calls = 0  
  
def fib_counted(n):  
    global calls  
    calls += 1  
    if n <= 1:  
        return n  
    return fib_counted(n - 1) + fib_counted(n - 2)  
  
for n in [10, 20, 25, 30, 32, 35]:  
    calls = 0  
    value = fib_counted(n)  
    print(n, value, calls)
```

MEASURED ON THE LECTURE MACHINE

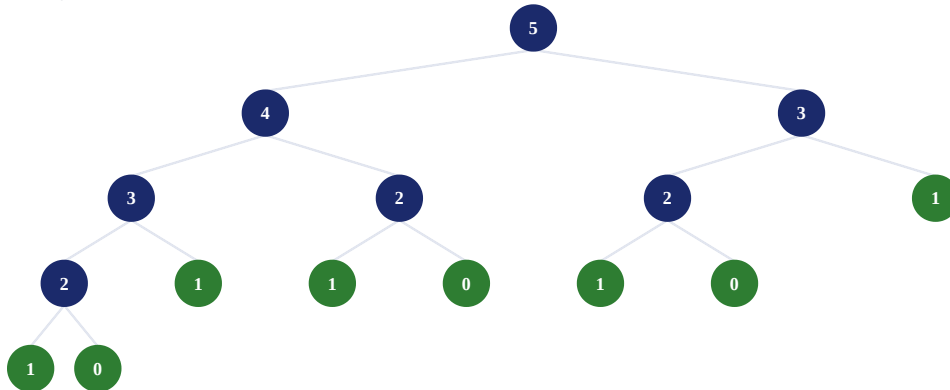
n	F(n)	calls	time (s)
10	55	177	0.000
20	6,765	21,891	0.001
25	75,025	242,785	0.013
30	832,040	2,692,537	0.146
32	2,178,309	7,049,155	0.362
35	9,227,465	29,860,703	1.453

Thirty million function calls to produce a seven-digit number. Apply the reaction test from Session 2, but with care: here the meaningful step is **adding one to n** , not doubling it. From $n = 30$ to $n = 32$ the count goes $2,692,537 \rightarrow 7,049,155$, a factor of 2.6 for two extra units — about 1.618 per unit.

THAT NUMBER IS NOT A COINCIDENCE

1.618 is the golden ratio ϕ . The number of calls is exactly $2 \cdot F(n+1) - 1$, and the Fibonacci numbers themselves grow like ϕ^n . So the cost of computing $F(n)$ this way is proportional to $F(n)$ itself — the algorithm does as much work as the size of the answer, which for an exponentially large answer is an exponentially large amount of work.

fib(5) — every node is one function call



fib(3) computed twice · fib(2) three times · fib(1) five times · 15 calls in all

The call tree for fib(5). Every node is one function call; green nodes are base cases. The subtree rooted at fib(3) is computed twice, fib(2) three times, fib(1) five times. The recursion keeps re-solving subproblems it has already solved, and the waste compounds at every level. Fifteen calls to compute a number you could get with four additions.

1.1 The diagnosis, which matters more than the example

Disjoint subproblems — healthy

Sorting the left half and the right half of a list. Nothing computed for the left half is needed for the right. No work is ever repeated. This is **divide and conquer**, and it is the subject of the next three sessions.

Overlapping subproblems — diseased

fib($n-1$) and fib($n-2$) share almost everything. The recursion computes the shared part separately for each branch, then does it again one level down. This is what makes the tree exponential.

RECURSION IS NOT SLOW. RECOMPUTING IS SLOW.

The cure for overlap is to remember answers instead of recomputing them — one dictionary, one line of code, and fib(500) becomes instant. That cure is called **dynamic programming** and we give it three full sessions in December. Today the point is only to be able to *tell the two diseases apart*, because they have different treatments.

Write $T(n)$ for the running time of a function on an input of size n . A recursive function gives an equation for T in terms of itself, and you read it off the code in three steps:

- 1 Find the **base case** and write down its cost — almost always $T(\text{small}) = \Theta(1)$.
- 2 Find each **recursive call** and write its cost as $T(\text{the size of that call})$.
- 3 Add the cost of everything the function does **outside** the recursive calls.

2.1 One call, size $n-1$

PY EXAMPLE A

```
def total(A, n):
    """Sum the first n elements, recursively."""
    if n == 0:
        return 0
    return total(A, n - 1) + A[n - 1]
```

base case: constant work
one call of size n-1, plus 0(1)

$T(n) = T(n-1) + 1$ with $T(0) = 1$.

2.2 One call, size $n/2$

PY EXAMPLE B

```
def bsearch(A, lo, hi, x):
    """Binary search, recursively."""
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if A[mid] == x:
        return mid
    if A[mid] < x:
        return bsearch(A, mid + 1, hi, x)
    return bsearch(A, lo, mid - 1, x)
```

ONE call, on half the range
ONE call, on the other half

$T(n) = T(n/2) + 1$ Two calls appear in the text, but only **one of them ever runs**. Counting calls in the source rather than calls on the execution path is the most common translation error.

2.3 Two calls, size $n/2$, plus linear glue

PY EXAMPLE C

```
def divide_shape(A):
    """The shape of every divide-and-conquer sort."""
    if len(A) <= 1:
        return A
    mid = len(A) // 2
    left = divide_shape(A[:mid])      # call of size n/2
    right = divide_shape(A[mid:])     # call of size n/2
    return combine(left, right)       # linear work to combine: O(n)
```

$T(n) = 2T(n/2) + n$

The single most important recurrence in this course. We meet it again as merge sort on

23 October.

2.4 The general form

THE DIVIDE-AND-CONQUER RECURRENCE

$$T(n) = a \cdot T(n/b) + f(n)$$

a = how many subproblems · b = by what factor each is smaller · $f(n)$ = work to split and combine

Function	a	b	$f(n)$	Recurrence
binary search	1	2	1	$T(n) = T(n/2) + 1$
merge sort	2	2	n	$T(n) = 2T(n/2) + n$
tree traversal	2	2	1	$T(n) = 2T(n/2) + 1$
Karatsuba multiplication	3	2	n	$T(n) = 3T(n/2) + n$

Note that `total` and `fib` do not fit this form: they reduce by *subtraction* ($n - 1$) rather than by *division* (n/b). Both kinds occur, and they behave very differently — something Friday's session will make painfully clear.

Three translation errors to watch for

1. Counting calls that appear in the text rather than calls that execute (§2.2).
2. Forgetting the glue. `A[:mid]` *copies*, so it is $\Theta(n)$ work, not free — Session 5's table applies inside recursive functions too.
3. Omitting the base case. A recurrence without one has no solution; it is the anchor that makes the whole equation finite.

SESSION 8 TAKEAWAY

A recursive function is analysed by writing an equation for its own running time: read off the base case, the recursive calls, and the glue. Recursion on **disjoint** subproblems is the engine of the next three sessions; recursion on **overlapping** ones is the Fibonacci disaster, and it has a different cure. We can now write the equations. We cannot yet solve them — that is Friday.

IN-CLASS EXERCISES (15 min — work in pairs)

1. Write the recurrence for each: (a) one call on $n-2$ with constant glue; (b) one call on $n/3$ with linear glue; (c) four calls on $n/2$ with constant glue; (d) two calls on $n-1$ with constant glue.
2. Draw the call tree for `fib(6)`. How many nodes? Check against $2 \cdot F(7) - 1$.
3. Here is a recursive function that reverses a list. Write its recurrence — and be careful about the slice:

```
def rev(A): return A if len(A) <= 1 else rev(A[1:]) + [A[0]]
```
4. Classify as disjoint or overlapping: (a) merge sort; (b) computing the binomial coefficient $C(n, k) = C(n-1, k-1) + C(n-1, k)$ recursively; (c) binary search; (d) finding all permutations of a list.
5. Why does `fib` take *longer* than it takes to count to $F(n)$?

Homework • due Friday 16 October

1. Reproduce the `fib_counted` table for $n = 10, 20, 25, 30, 32, 35$. Verify empirically that the call count equals $2 \cdot F(n+1) - 1$ for every n you tested.
2. Compute the ratio of consecutive call counts for $n = 20 \dots 30$. How close do you get to $\phi = 1.618$?
3. Write an *iterative* Fibonacci using one loop and two variables. Time both versions at $n = 30$ and report the ratio. Which complexity classes are you comparing?
4. For each of these, write the recurrence and identify a , b and f where the form applies: binary search, merge sort's `divide_shape`, `rev` from Exercise 3, and a function making 3 calls on $n/4$ with $\Theta(n^2)$ glue.
5. Reading: CLRS §4.1 (the maximum-subarray problem) — read only as far as the recurrence, and write down what a , b and f are for it.

Next session • Friday 16 October: Solving recurrences by unrolling. We substitute the equation into itself, watch the pattern emerge, and derive the cost of insertion sort and of the Towers of Hanoi from scratch.

